

Karla Alvarez

5/12/2026

CSN 150: Final Project

Storm-Breaker: A Social Engineering Tool

Social engineering remains one of the most effective attack vectors in cybersecurity, exploiting human psychology rather than technical vulnerabilities. Unlike traditional penetration testing that targets software flaws or missed patches, social engineering attacks rely on the manipulation of people to access sensitive information or performing actions that may compromise security. This report documents the evaluation and deployment of the Storm Breaker, a social engineering tool, within a controlled setting.

Storm Breaker provides **six primary attack vectors** for simulated social engineering campaigns:

- **Webcam access:** Captures images and device information when targets grant camera permissions
- **Location tracking:** Harvests GPS coordinates through deceptive web interfaces
- **Microphone access:** Records audio from the target device
- **OS password grabber:** Tricks targets into entering their operating system credentials
- **Device information:** Collects hardware and system data
- **Ngrok link generation:** Creates both worldwide and localhost URLs for flexible deployment

This tool is designed to facilitate a phishing simulation through a payload delivery. All testing conducted in this project have been tested in a controlled, isolated environment, so this is a walkthrough of the steps I took.

This project was practiced on my Kali Linux virtual machine. Storm Breaker requires following programs to run properly :

- php
- python3
- git
- Ngrok

I first cloned the Github repository into a folder on my directory using this command:

```
(kali@kali)-[~/Projects/stormbreaker/Storm-Breaker]
$ ls
install.sh  README.md  Settings.json  st.py
modules    requirements.txt  storm-web

(kali@kali)-[~/Projects/stormbreaker/Storm-Breaker]
$ sudo bash install.sh
```



```
git clone https://github.com/ultrasecurity/Storm-Breaker
```

This downloads the source code from the original Github repository into my local machine. The cloned directory includes:

- **Source code files** containing the tool's core functionality
- **Configuration scripts** for system integration
- **Installation files** (install.sh) for automated setup
- **Dependency lists** (requirements.txt) specifying Python packages needed
- **Documentation files** explaining tool usage

I then changed my directory into the folder where the source code should be and installed the installation script with root privileges:

```
cd Storm-Breaker
```

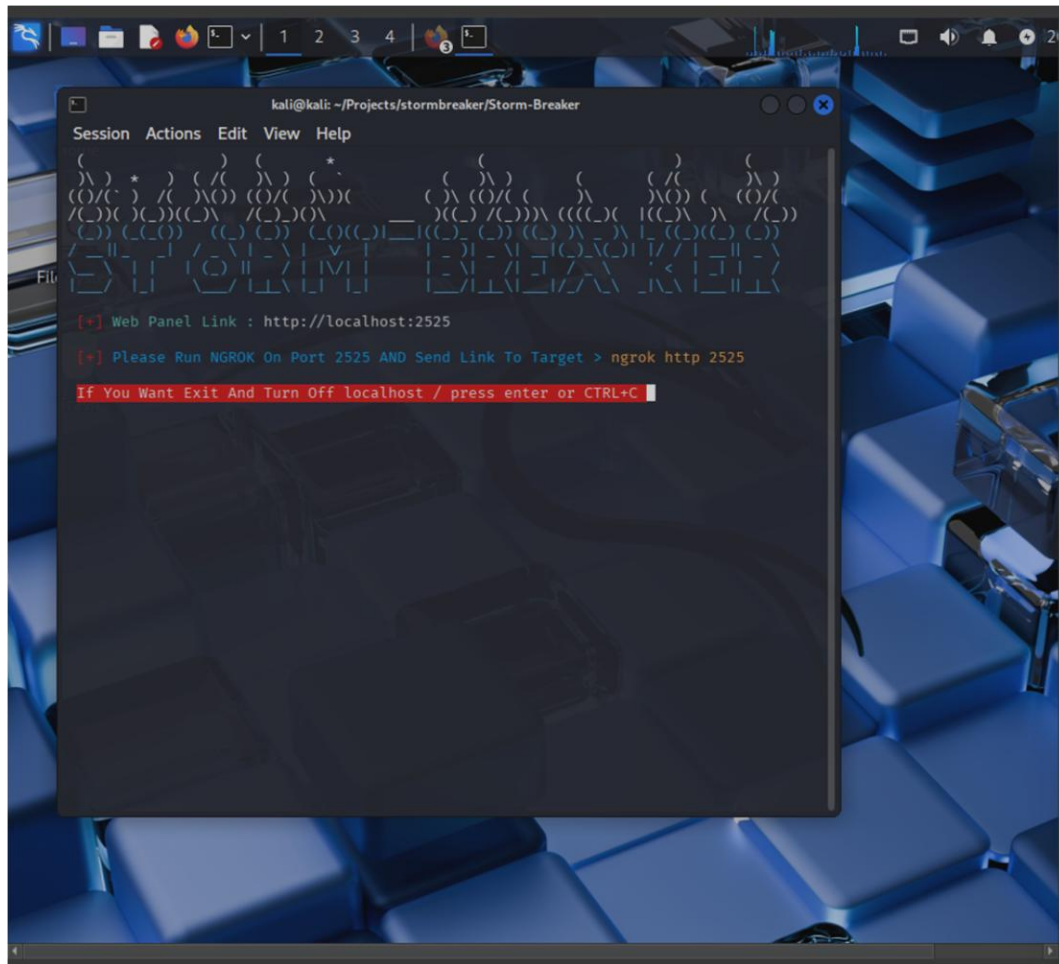
```
sudo bash install.sh
```

This command configures the required libraries to execute files, installs missing dependencies, and sets permissions to execute this tool seamlessly. I also installed the Python package dependencies and libraries that Storm Breaker relies on.

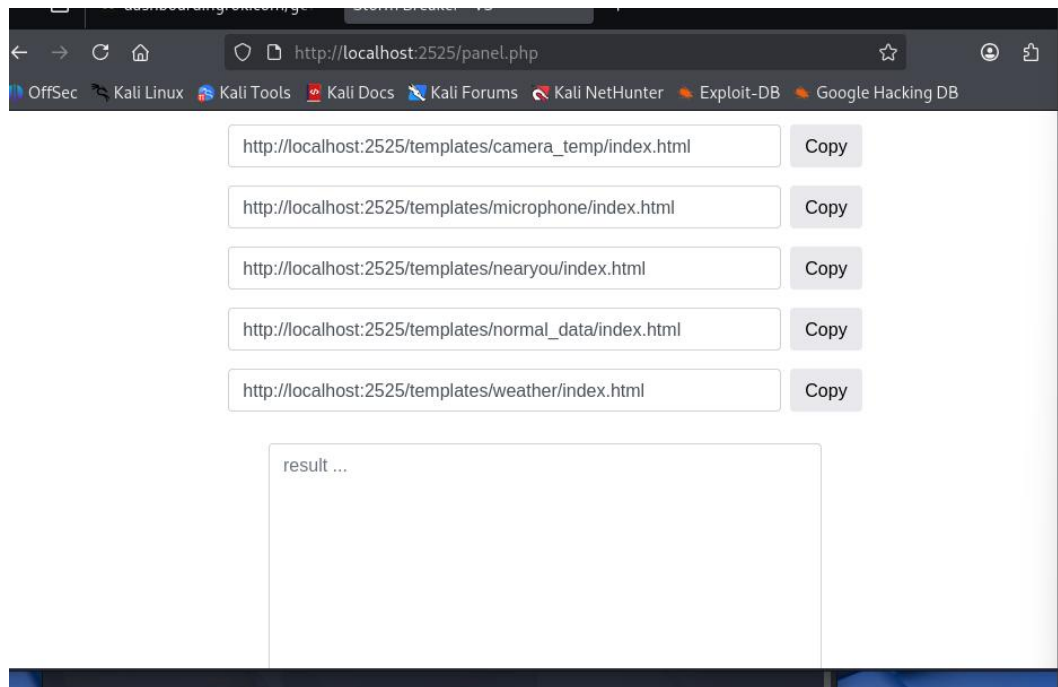
```
sudo python3 -m pip install -r requirements.txt
```

And now, I launch the Storm Breaker application using Python 3, initializing the tool and launching the web server and gives me a clickable link that is opened below:

```
sudo python3 st.py
```

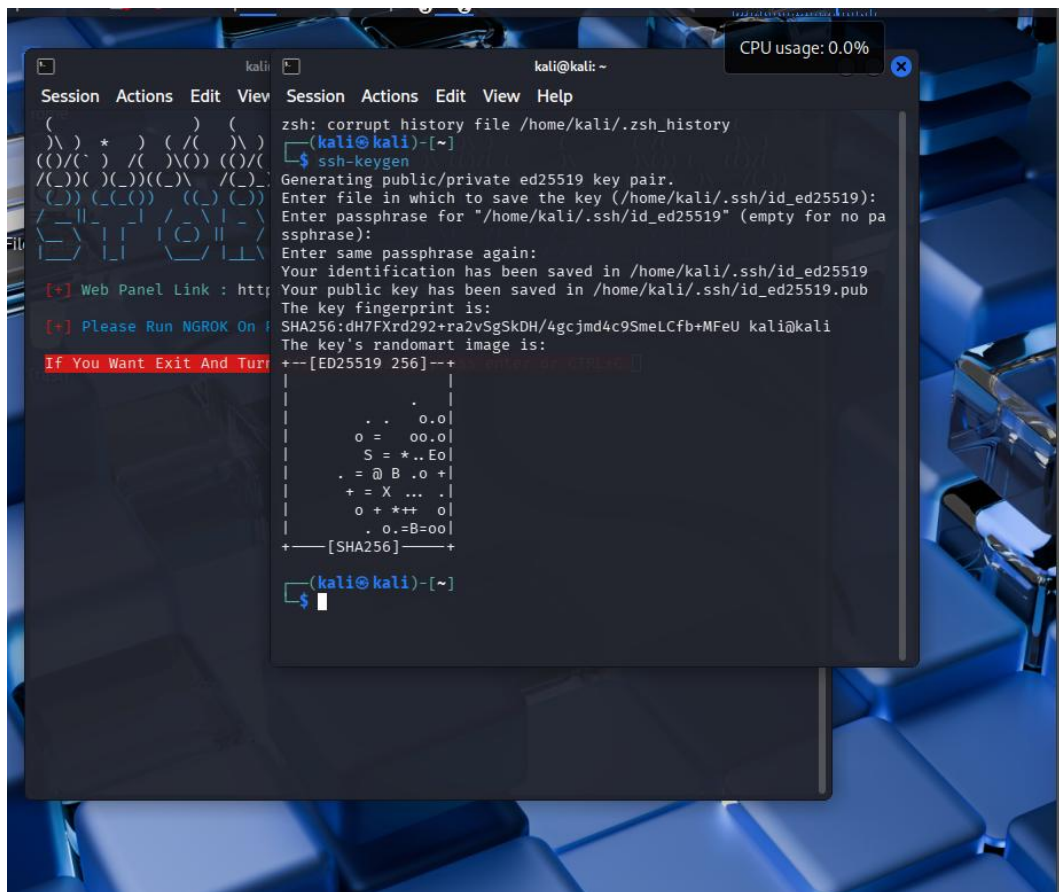


When successfully launched, this opens the application banner along with the web page to payloads. It web page takes us to a login page. Now when logged into with the default username and password, the payloads are loaded as links.



With the steps I was provided, I was recommended to generate an SSH key pair for authentication after generating a token. This allows authentication for SSH connections.

ssh-keygen



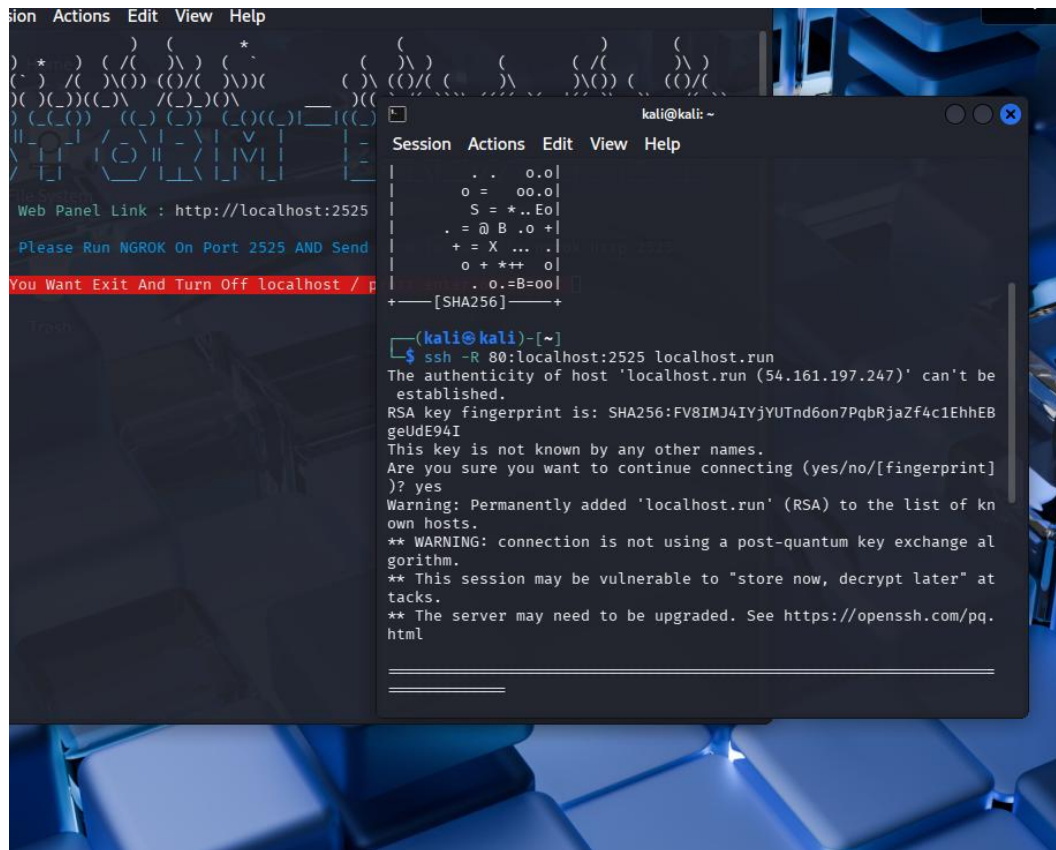
I then created a tunnel to expose my local server to the Internet:

```
ssh -R 80:localhost:2525 localhost.run
```

For more context of this command:

- ssh - Secure Shell protocol client
- R 80:localhost:2525 - Reverse port forwarding with three parts:
 - 80 - Remote port (the port on the localhost.run's server)
 - localhost – My local machine
 - 2525 - Local port where Storm Breaker server is running
 - localhost.run - The remote tunneling service

This allows traffic from the Internet to my remote port, and into my local machine port. So, victims of this attack would access the public URL and interact with my local server.



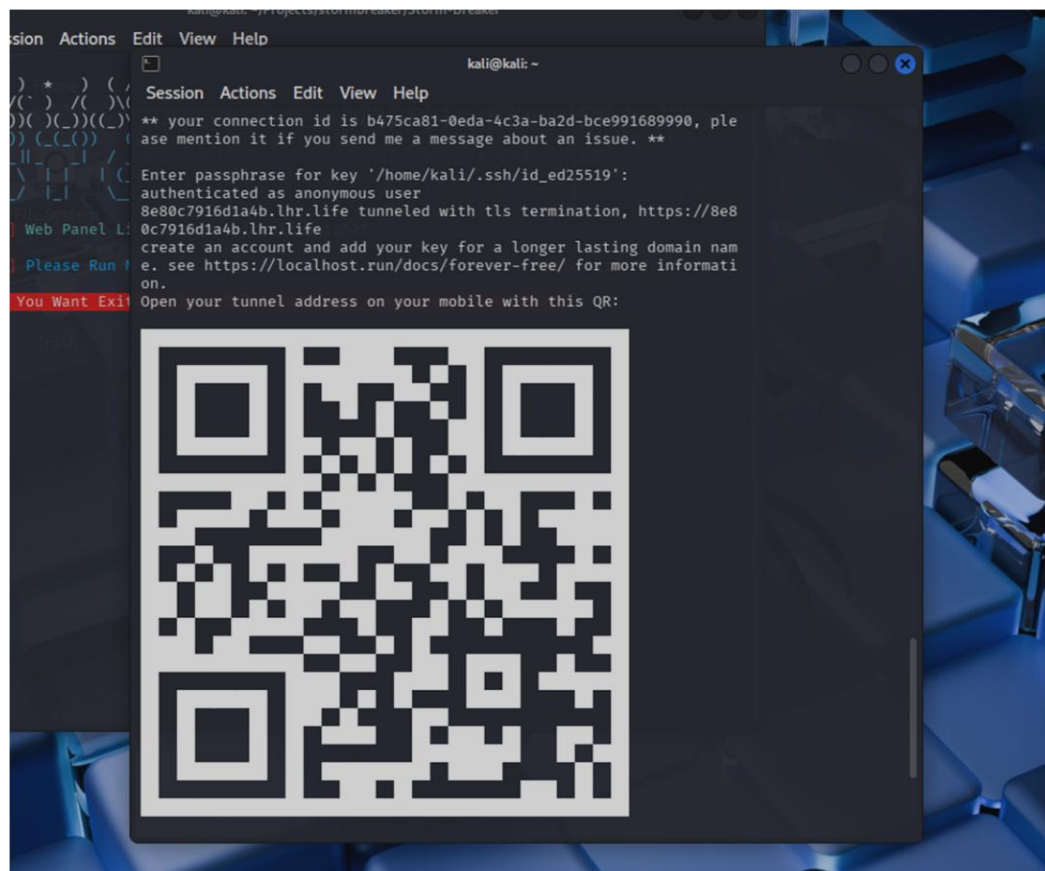
The image shows a Kali Linux terminal window with a dark background and blue text. The window title is 'kali@kali: ~'. The terminal displays the following content:

```
Session Actions Edit View Help
. . o.o|
o = oo.o|
S = *..Eo|
. = @ B .o +|
+ = X ... .|
o + *++ o|
. o.=B=oo|
+---[SHA256]---+

(kali@kali)-[~]
$ ssh -R 80:localhost:2525 localhost.run
The authenticity of host 'localhost.run (54.161.197.247)' can't be
established.
RSA key fingerprint is: SHA256:FV8IMJ4IYjYUtd6on7PqbRjaZf4c1EhhEB
geUdE94I
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint]
)? yes
Warning: Permanently added 'localhost.run' (RSA) to the list of kn
own hosts.
** WARNING: connection is not using a post-quantum key exchange al
gorithm.
** This session may be vulnerable to "store now, decrypt later" at
tacks.
** The server may need to be upgraded. See https://openssh.com/pq.
html
```

In the background, another terminal window is partially visible, showing a 'Web Panel Link : http://localhost:2525' and instructions to 'Please Run NGROK On Port 2525 AND Send' and 'You Want Exit And Turn Off localhost / p'.

It then generated this QR code below for scanning:



With authorized testing, I scanned it with my target's device and logged into the URL generated in my virtual machine's terminal to execute specific payloads. Storm Breaker serves the phishing page and captures the data of my target device as the data appears on my terminal in real time. Some tips include the following:

- **Keep the tunnel running** - If you close the terminal or stop the SSH connection, the URL dies
- **Test it yourself first** - Open the URL in your browser to verify the page loads correctly
- **Document everything** - Screenshot the setup and results for your penetration testing report
- **Authorized testing only** - Ensure you have written permission and document the scope

This penetration tool demonstrates the operation of Storm Breaker, a toolkit used to simulate phishing attacks and assess organizational security awareness, along with the security of personal camera devices. Getting a better comprehension of how these kinds of attacks are generated, deployed, and executed helps build effective defensive strategies. And most importantly, it puts a lens on the ethical responsibility that comes with security knowledge. The ease with which these

tools can be deployed underscores the critical importance of obtaining proper authorization, maintaining clear scope boundaries, and using these capabilities exclusively for defensive improvement rather than malicious purposes.

A huge part of cybersecurity is not only understanding the technical components but also how human behavior is studied and used. The ultimate takeaway is that a huge part of security control is human knowledge of how these attacks look like and how to defend against it. While investing in protective and defensive software, continuous education holds weight just as much as the technological solutions.